

Prototyping Use as a Software Requirements Elicitation Technique: A Case Study

Dante Carrizo(✉) and Iván Quintanilla

Computer Science Department, University of Atacama, Copiapó, Chile
{dante.carrizo,ivan.quintanilla}@uda.cl

Abstract. Prototyping is an agile software development methodology. It has also been proposed as a technique to obtain the software requirements from the stakeholders. However, there are few publications proposing a prescriptive guide and show its use in practice. This article presents a report of a case in which the prototype technique was used to elicit requirements of a software system in the university academic context. For this, authors propose a procedure to apply and they carry out elicitation sessions with two stakeholders who have different familiarity with the domain. The results show that the technique is effective in achieving a high coverage of the requirements and that it seems to perform better with stakeholders who have more familiarity with the domain. Although the results do not have statistical power, the case yields trends that can help development teams adopt this technique to produce the requirements in certain cases.

Keywords: Requirements engineering · Prototyping technique
Familiarity with the domain · Requirements production · Case study

1 Introduction

Software engineering is an engineering discipline that is concerned with all aspects related to the production of software [1], so it suggests the application of a systematic, disciplined and quantifiable approach to the development, operation and maintenance of software [2]. According to Pressman [3], software engineering is a technology composed of layers: Tools, Methods, Process and Commitment to quality. The process layer is the basis of software engineering that holds together the technological layers, and rationally and timely enables the development of computer programs. There are several methodologies to carry out this process. These are grouped or classified in Classical Methodologies, such as Cascade, V Method, Spiral, among others, and Agile Software Development Methodologies such as Extreme Programming, Scrum, and Prototyping, among others.

Regardless of the methodology to be followed for development, there are cross-sectional stages of the process to be carried out, such as Requirements, Design, Implementation, Testing and Maintenance. The requirements stage corresponds to the process of understanding and defining the services or functions of the system and the restrictions on its development [1]. This process of requirements is broken down into stages such as Elicitation, Analysis, Documentation and Validation [4].

© Springer International Publishing AG, part of Springer Nature 2018
Á. Rocha et al. (Eds.): WorldCIST'18 2018, AISC 746, pp. 341–350, 2018.
https://doi.org/10.1007/978-3-319-77712-2_33

Software prototyping has been used as an agile development methodology [5–9], but has also been used as a technique to produce software requirements [10–14, 21]. In scientific literature, however there is little evidence of the use of this technique in elicitation. This is because there is little formalization of the procedure to apply it. Apparently, the technique is usually ideal when stakeholders have difficulty in externalizing the needs [15].

This research focused on deepening the use of the prototyping technique in a real development project. For this, the technique was applied in the development of a system for monitoring curricular redesigns of a Chilean university. Preliminarily, the authors had to propose a prescriptive procedure of the technique. Subsequently, elicitation sessions were held with two stakeholders with different knowledge of the domain. The requirements capture with both stakeholders was contrasted with a Gold Standard, a list of requirements obtained with an expert in the domain. In this way, the coverage of requirements obtained from both types of stakeholders could be known and compared.

The rest of the document continues with Sect. 2, where some works related to this research and that motivate its development, are presented. Section 3 describes the research methodology to follow to carry out the study. Section 4 explains in detail the actions performed in each of the stages of this study. Section 5 summarizes the interpretation and impact of the results obtained based on this research.

2 Work Related

In the scientific literature, there are works that use prototyping as a technique for eliciting software requirements, which are classified into two groups: theoretical and practical. In the first group, there are those works that propose the use of prototyping as a technique for eliciting requirements, only at a descriptive level. Those of the second group, on the other hand, focus on the application of the technique in the elicitation of system requirements where the use of the technique is shown.

Among the theoretical works is Bahn and Naumann [11]. They explains the concept of prototyping, the types of prototypes and their use. However, there is no prescriptive guide or procedure for use. In the same sense, Cho et al. [13] use software prototypes to produce requirements from stakeholders, but it is presented in a descriptive way, that is, it lacks a formal prescriptive method. On the other hand, Asur and Hufnagel [10] describe a procedure for the use of the technique, but do not present an application of the proposed procedure. Finally, Vijayan and Raju [12] describe a set of steps to apply the technique and also present a case study where they use the technique, however, it does not describe it in sufficient detail.

In the second group, Mannio and Nikula [14] propose a procedure and apply prototyping as an elicitation technique to a toy case, that is, it is not a real case of elicitation of software requirements.

In summary, the literature makes mention of prototyping as a technique elicitation of software requirements both theoretically and practically. However, it does not delve into its prescription, which is the reason for this research.

3 Research Methodology

The prototyping has been used as an agile software development methodology, as well as a software requirements elicitation technique, but in the latter case, will it be an effective technique? Will stakeholder knowledge of the problem domain influence its effectiveness?

For this, the question to answer in this investigation is:

How effective is the prototype technique in the elicitation of software requirements from stakeholders with and without familiarity with the domain?

To answer this question, this investigation was carried out following the steps shown in Fig. 1 and described below.

- Definition of the case study: The case study is aimed at the development of a software system framed in the university academic field. For this purpose, three individual elicitation sessions are carried out with three different stakeholders. In each of the sessions a list of requirements is obtained.
- Proposed procedure: The description of the procedure proposed by the authors is presented to apply the prototyping technique in software requirements elicitation.
- Obtaining the gold standard: An elicitation session of software requirements (S1) is carried out with an expert stakeholder in the domain of the problem. The requirements obtained in this session are considered as an ideal list of requirements for the software system, called by the authors as Gold Standard.
- Sessions with stakeholders: Two sessions of elicitation are held. In one of them (S2) the stakeholder has a low level of knowledge of the problem domain, and in the other (S3) the stakeholder has a high level of domain knowledge. For each of the elicitation sessions, the requirements obtained are listed in separate lists.
- Discussion of results: The requirements obtained in sessions S2 and S3 are compared with the requirements obtained in the Gold Standard in order to obtain the degree of requirements coverage through the application of the technique considering the difference between the stakeholders considered for both sessions.



Fig. 1. Research method.

4 Development of the Case Study

4.1 Definition of the Case Study

For more than a decade, universities both in Chile and around the world have felt the need to change their curricula from a traditional curriculum, centered on teaching, to a curriculum with a focus on competences, focused on learning. This involves a process of reviewing their study plans in order to design or redesign them taking into account

this paradigm shift, which involves a work of gathering and analyzing information from various sources both inside and outside the university. Performing this process manually, without the help of a computer tool, can cause losses or inconsistencies in the information and as a consequence a delay in the continuous improvement of the same. Due to this, there is a need to have a computer tool. A software system that allows storing and analyzing all the design information or redesign of the curricula based on a methodology with focus on competencies.

The software system to be developed consists of a web platform accessible from any computer connected to the Internet, both inside and outside the university. The platform should allow users to design, monitor and evaluate curricular designs of careers within the university by implementing a curricular design methodology with a focus on competencies.

Only those who have a user account, that is, who are previously registered in it, can access this platform. The user accounts have user profiles assigned to them as administrator, academic and academic coordinator that establish a set of options and restrictions to use the platform according to their user profile. In addition, each user is associated with one of the various careers offered by the university. The administrator can see the information of all the curriculums, for all the careers of the university that are entered in the platform. The academic coordinator can modify the curricula that belong to the associate career. The academic can only see the information of the curricula that belong to the career that is associated.

One of the main functions of the platform is the monitoring of the curriculum through an interactive system of views both general (upper view) and specific (lower view) of the curricular design defined for a career. The general view, allows graphically visualize the information of each of the elements that make up the curricular design such as Units of competence, Elements of competence and Subjects and their relationship between them. This allows for a follow-up in order to evaluate aspects such as the subjects related to a unit of competence, which is not being achieved, elements of competence that are not being implemented or achieved in one or more subjects, etc. Each time the user interacts with any of these elements, the rest that are related to it, will be highlighted with a distinctive color in order that the user graphically visualize the relationships between them. The specific view allows you to visualize for each subject the relations between the elements of the curricular design as Associated Units and Competency Elements, but also others that are specific to the subjects, such as Learning Units, Learning Results, Evaluation Instruments, Teaching methods, among others. All this allows for a follow-up in order to evaluate aspects such as the competences achieved by the students, the performance of the students, the evaluation instruments to measure the achievement of the learning results, and the learning methodologies used for the achievement of the same, etc.

In order for this graphical visualization to be carried out, it is necessary to be able to manage all the information related to the curricula of the careers. For this, the academic coordinator will have access permissions to enter, modify and eliminate the information through web forms for the entry of information. However, he or she will only be able to manage the information of the curriculums that belong to the career that is associated. He will also be able to use the functionality of views for the curricular designs he has

entered on the platform. Along with the input of information and views of the platform, the software system should allow generating a document with all the information entered on a curricular design of a career.

The academic coordinator will be able to manage the curricular designs, access the system of views and the generation of the document for the curricular designs of the career to which he is associated. Finally, the academic profile can only access the system of views and the generation of the curricular design document.

4.2 Prototyping Procedure

In the literature, various prototyping procedures appear, but they are not formally defined. That is the reason why the authors propose a prescriptive procedure shown in Fig. 2. For reasons of space, the procedure detail of the prototyping technique can be seen in http://www.diicc.uda.cl/magister/documentos/prototyping_procedure.pdf.

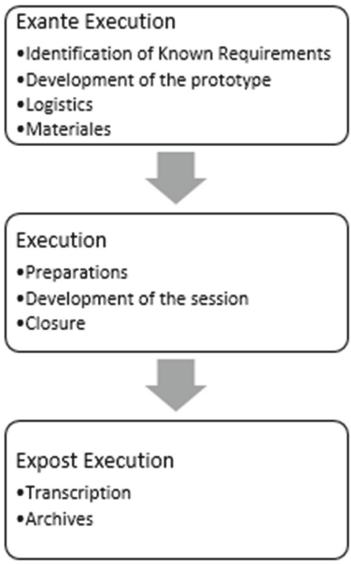


Fig. 2. Proposed prescriptive procedure for prototyping technique.

4.3 Obtaining Gold Standard

The objective of the case study is to use prototyping as a technique to elicit software requirements for the development of the platform above described. In order to compare the effectiveness of the technique, a requirements elicitation session was carried out with an expert stakeholder in the problem domain from which an ideal list of requirements for the platform named Gold Standard was obtained. The expert knows in depth the trend of curricular innovation, which takes more than a decade, understands the underlying concepts of curricular design with a focus on competencies. He knows the

curricular redesign processes carried out both in Europe and in Latin America, possesses knowledge and skills on the curriculum redesign work, where he has participated in various curricular redesign processes both in Chile and abroad, so he knows and understands the need for a computer tool to help in the curricular redesign process.

The elicitation session with the expert lasted about 1 h. This session was carried out according to the protocol defined in the prescriptive procedure proposed for the use of the technique. As outcome of this session, a total of 74 requirements that make up the Gold Standard were obtained. These requirements are considered ideal for the platform, since they were elicited from an expert stakeholder in the domain of the problem. Later, this Gold Standard will be compared with the list of requirements obtained in sessions S2 and S3 in order to evaluate the effectiveness of the prototyping technique.

4.4 Eliciting Requirements

Once Gold Standard is obtained, sessions S2 and S3 are held in which two representative stakeholders of two different types of stakeholders are considered. The first one corresponds to a full-time academic and researcher of the university. This stakeholder has previously performed academic coordination tasks, but with the classic curriculum approach, that is, focused on teaching and content. He knows little about curricular design with a focus on competencies. He has heard about it, but has not deepened much on the subject, since it has no greater responsibility than working based on the guidelines of the current academic coordinator, who leads the process of curricular redesign by competencies for the race. However, he contributes with his knowledge and experience as an academic and researcher in the process of curricular redesign of the career. The second stakeholder is an academic and researcher of the full-time university, but unlike the first one, he has a deep knowledge of the subject, because he is the academic coordinator and who leads the curricular redesign process; he is the one who is permanently aware of all the information of the curricular redesign of the race. Along with leading the redesign process within the career is who presents the information to the higher academic bodies so it also handles their feedback. Next, the procedure of Fig. 2 is described in detail, carried out in the elicitation sessions of requirements carried out with both stakeholders.

ExAnte Execution

Logistics: For the elicitation session by prototyping, three stakeholders are considered, described below. The sessions were planned with a duration of 1 h approx. and they were scheduled keeping in mind the availability of time between the elicitor and the stakeholders, since the sessions were held in the office of each stakeholders.

Materials: The materials used in the session were Video camera (with tripod), Audio recorder (Cell), Notepad and pencil, and Notebook (Prototype and stopwatch).

Preparation: Before starting the session, it should be established where the stakeholder and the computer will be located. The location should be a comfortable and relaxed place to work; it can be your own desk or locate a second desk or table for the session. Then, the video camera is mounted to the tripod and it is located in a position that delivers a plane that visualizes both the notebook screen and the stakeholder, this mainly to obtain

information about non-verbal language. The audio recorder is located in a place that does not obstruct the stakeholder's work during the session. Once all the above is finished, the timer starts and the recording of both video and audio begins.

Execution
The educator begins the session by greeting the stakeholder, mentions his name. He indicates that he is the one who will elicit the requirements based on the needs expressed by the stakeholder; he declares that the session aims to obtain requirements for a professional platform using the prototype as support. Next, the elicitor presents the prototype functionalities one by one and asks the stakeholder to execute them. The stakeholder can ask questions or comments, if any, about what is happening while he is operating the prototype.

The elicitor begins by presenting to the stakeholder the start screen of the prototype where the authentication of the system is located, that is, in order to use the prototype the stakeholder must have a user account (username and password) registered in the system. The elicitor explains to the stakeholder the different user profiles that exist and the information to which they can access, such as that their profile allows them to manage (add/edit/delete) various curricular designs, but only those that belong to the race associated with the user.

The elicitor, in a first part of the session, describes the data entries of the prototype, explains how it can manage and complete the information of a curricular design (through a system of tabs), that is, how to complete the forms for all the parts (components) that the curricular design has, save the changes, etc. Previously, all the information regarding a curricular design has been entered in order to be able to cover other features of the prototype.

In a second part of the session, the elicitor presents to the stakeholder what the prototype produces as output. On the one hand, a graphic panel is presented where the main components of a curricular design appear (Competency Units, Competency Elements and the Curricular Mesh). It is explained to the stakeholder that the graphic panel aims to understand, in a graphic visual way, how these components are related. As the elicitor explains the functionalities, the stakeholder comments and makes suggestions about everything that is identified as failure, functionality difference or lack of information.

When all the functionalities are reviewed, the elicitor concludes with a summary of the observations obtained in the session, and stops the audio and video recording and the stopwatch.

Closing: At the moment when everything stops, elicitor thanks the stakeholder for the time and disposition granted, keeping all the equipment and leaving everything as it was when it arrived at the place.

Expost Execution
Transcription: The elicitor transcribes both video and audio recordings, taking into account that they are as faithful as possible to the recording.

Archiving: The transcription of the recording is saved for future use, which can be used to build a Requirements Specification or use it as an input for subsequent sessions.

4.5 Discussion of Results

The requirements obtained in each elicitation sessions are summarized in Table 1.

Table 1. Summary of requirements obtained in elicitation sessions.

Session	Domain knowledge level of stakeholder	Quantity of requirements captured	Coverage percentage
Gold standard	Expert	74	100%
S2	Low	14	19%
S3	High	31	42%
S2 + S3		45	61%

As shown in Table 1, in the elicitation session with the expert stakeholder, a total of 74 requirements were obtained. Then, in the requirements elicitation session S2 (with the stakeholder with low level of knowledge of the problem domain) 14 requirements were obtained, and in the requirements elicitation session S3 (with the stakeholder with high level of knowledge of the problem domain) 31 requirements were obtained. In terms of coverage, that is, by comparing the requirements obtained in the Gold Standard with those obtained in each elicitation sessions, they represent coverage of 19% and 42% respectively. In addition, adding the requirements obtained in sessions S2 and S3 results in a total of 45 requirements, which represents a 61% coverage of the requirements obtained in the Gold Standard.

However, the results obtained may not be entirely statistically conclusive. Even so, based on the requirements obtained in S2 and S3, the number of requirements elicited from the stakeholders with a high level of knowledge of the domain (nS3) is greater in comparison to the stakeholder with low level of domain knowledge (nS2), i.e. that, $nS3 > 2nS2$, with which it can be conjectured that the technique is more effective to elicit requirements from a stakeholder with a high level of knowledge of the problem domain. On the other hand, by meeting the requirements elicited in both sessions, it is possible to cover more than half of the Gold Standard, which shows a tendency to say that prototyping is effective as a technique for eliciting of software requirements.

The foregoing points to the fact that knowledge of the domain positively influences the elicitation of requirements from the stakeholders, that is, it can capture a greater number of requirements from a stakeholder that has knowledge of the domain versus one that does not have knowledge of the domain. However, this contrasts with that posed by various authors who point out that the lack of knowledge or ignorance of the domain can help to obtain more requirements from the stakeholders. In this sense, Berry [16] states that knowledge of the problem domain can lead to unspoken assumptions by failing to probe what seems obvious. According to Berry, lack of domain knowledge brings benefits to requirements engineering activities such as generating ideas and asking questions that experts in the domain overlook. On the other hand, Dieste [17] concluded that the elicitor's experience, using the interview as a requirement elicitation technique, does not affect the amount of ideas generated during the interview. Kenzi [18] studied the effect of domain knowledge by conducting interviews and various techniques in the process of eliciting requirements, and determined that without knowledge of the

domain they can be more effective. Schooenderwoert [19] reported a case study in which he compared the data obtained from an agile development team composed of novices against a number of industrial projects. This comparison showed that the team composed of novices obtained better performance than other teams that performed the same job. Finally, Kristensson [20] studied the generation of ideas for a problem on the mastery of mobile technology using three types of stakeholders: ordinary users who did not study computer science, advanced users who studied computer science and professional developers of software products. Ordinary users were considered as ignorant in the domain and advanced users and developers as experts in the domain. The data showed that the ideas generated by ordinary users were more valuable than those developed by advanced users and developers, but, on the other hand, the ideas of advanced users and developers were more feasible than those raised by ordinary users.

Finally, responding to the research question, we can indicate that, although the results have little statistical power, the prototyping technique tends to be more effective to capture requirements from a stakeholder who has knowledge of the domain than the one who does not have or have a low level of domain knowledge.

5 Conclusions

This study carried out the elicitation of software requirements for the development of a university academic platform, from two stakeholders with different level of knowledge of the problem domain using the prototyping technique. In addition, in order to determine the effectiveness of the technique, the results obtained from the stakeholders were compared with an ideal list of requirements (gold standard) obtained from an expert stakeholder.

Because the procedures on the use of prototyping to elicit requirements found in the literature do not include sufficient level of detail, this study presents a practical guide proposing a prescriptive procedure of the technique in order to help the requirements engineers on the use of the technique. In addition, this research explains in detail the use of the prescriptive prototyping procedure as a technique for eliciting software requirements applied to a real case of software development.

Even when there is a threat to external validity, that is, generalization of results, they reflect that on the one hand the technique is effective to capture software requirements, and that this effectiveness is greater when the stakeholder has a high level of knowledge of the problem domain to tackle. The latter, contrary to that thrown by other studies, can be explained in the relation of the nature of the technique that requires knowledge of the domain to generate it and use it.

As future work, an experiment with greater participation of subjects can be carried out to validate this exploratory study.

Acknowledgements. This work was funded by the Research fund of the University of Atacama, project DIUDA 22298.

References

1. Sommerville, I.: Ingeniería de Software. Naucalpan de Juarez, México. Pearson Education (2011)
2. Radatz, J., Geraci, A., Katki, F.: IEEE standard glossary of software engineering terminology. IEEE Std, 610121990(121990) (1990)
3. Pressman, R.S.: Software Engineering: A Practical Approach. McGraw-Hill, New York (1988)
4. Kotonya, G., Sommerville, I.: Requirements Engineering: Processes and Techniques. Wiley, Chichester (2002)
5. Andriole, S.J.: Fast, cheap requirements prototype, or else! IEEE Softw. **11**(2), 85–87 (1994)
6. Carey, J.M.: Prototyping: alternative systems development methodology. Inf. Softw. Technol. **32**(2), 119–126 (1990)
7. Jones, T.S., Richey, R.C.: Rapid prototyping methodology in action: a developmental study. Educ. Technol. Res. Dev. **48**(2), 63–80 (2000)
8. Berzins, V.: Rapidly prototyping real-time systems. IEEE Softw. **5**(5), 25–36 (1988)
9. Ollila, E.M., Suomela, R., Holopainen, J.: Using prototypes in early pervasive game development. Comput. Entertainment (CIE) **6**(2), 17 (2008)
10. Asur, S., Hufnagel, S.: Taxonomy of rapid-prototyping methods and tools. In: Proceedings of Fourth International Workshop on Rapid System Prototyping. Shortening the Path from Specification to Prototype, pp. 42–56. IEEE (1993)
11. Bahn, D.L., Naumann, J.D.: Evolution versus construction: distinguishing user review of software prototypes from conventional review of software specifications. In: Proceedings of the 1997 ACM SIGCPR Conference on Computer Personnel Research, pp. 240–245. ACM (1997)
12. Vijayan, J., Raju, G.: Requirements elicitation using paper prototype. In: Advances in Software Engineering, pp. 30–37 (2010)
13. Cho, H., Kinny, D., Lin, D.: Parallel prototyping for multi-language service design: a case study on introducing a multilingual tool into a Japanese local restaurant. In: 2013 International Conference on Culture and Computing (Culture Computing), pp. 86–91. IEEE (2013)
14. Mannio, M., Nikula, U.: Requirements elicitation using a combination of prototypes and scenarios. In: WER, pp. 283–296, ISO 690 (2001)
15. Abran, A., Moore, J.W.: SWEBOK: Guide to the Software Engineering Body of Knowledge. IEEE Computer Society, California (2004). Ed. 2004
16. Berry, D.M.: The importance of ignorance in requirements engineering. J. Syst. Softw. **28**(2), 179–184 (1995)
17. Dieste, O., Juristo, N., Shull, F.: Understanding the customer: what do we know about requirements elicitation? IEEE Softw. **25**(2), 11–13 (2008)
18. Kenzi, K., Soffer, P., Hadar, I.: The role of domain knowledge in requirements elicitation: an exploratory study. In: Proceedings of the Fifth Mediterranean Conference on Information Systems (MCIS) (2010)
19. Van Schooenderwoert, N.: Embedded agile project by the numbers with newbies. In: Proceedings of the AGILE Conference (AGILE), pp. 351–366 (2006)
20. Kristensson, P., Gustafsson, A., Archer, T.: Harnessing the creative potential among users. J. Prod. Innov. Manag. **21**(1), 4–14 (2004)
21. Rocha, Á., Vasconcelos, J.B.: A framework to analyse the approach adopted in the information systems requirements engineering activity. In: Proceedings of the SERP04 - The 2004 International Conference on Software Engineering Research and Practice, Las Vegas, USA (2004)